Testbench Modeling in Verilog

RTL Design and Verification Fundamentals

October 12, 2026

Contents

1 Introduction **2**

2 Basic Testbench Structure **2**
2.1 General Template 2

3 DUT Instantiation **3**
3.1 Example: Adder DUT 3

4 Generating Input Stimulus **3**
4.1 Example 3

5 Complete Example: Adder Testbench **4**
5.1 DUT Code 4
5.2 Testbench Code 4
5.3 Expected Results 5

6 Clock Generation **5**
6.1 Example 5

7 Reset Generation **5**
7.1 Example: Synchronous Reset Stimulus 6

8 Monitoring Simulation Results **6**
8.1 `$display` 6
8.2 `$monitor` 6
8.3 `$finish` 6

9 Self-Checking Testbench **7**
9.1 Example 7

10 Blocking and Nonblocking Assignments **8**

11 Testbench Modeling Guidelines **8**

12 Summary **9**

1 Introduction

A testbench is a simulation environment used to verify the functional behavior of a Design Under Test (DUT).

The DUT may be a combinational circuit, sequential circuit, controller, or complete RTL module.

A testbench generates input stimulus, observes outputs, and checks whether the DUT behaves as expected.

A testbench is generally simulation-only and is not intended for hardware synthesis.

2 Basic Testbench Structure

A typical Verilog testbench contains:

1. Signal declarations for DUT inputs and outputs.
2. Instantiation of the DUT.
3. Input stimulus generation.
4. Clock and reset generation, when required.
5. Output monitoring and result checking.
6. Simulation termination.

2.1 General Template

```
module tb_example;

    reg  a;
    reg  b;
    wire y;

    // DUT instantiation
    design_module dut (
        .a(a),
        .b(b),
        .y(y)
    );

    // Input stimulus
    initial begin
        a = 1'b0;
        b = 1'b0;

        #10 a = 1'b1;
        #10 b = 1'b1;

        #10 $finish;
    end
```

```
24
25 endmodule
```

The testbench drives the DUT inputs and observes the output `y` during simulation.

3 DUT Instantiation

The Design Under Test is instantiated inside the testbench just like a module in a hardware design.

3.1 Example: Adder DUT

```
1 module adder (
2     input  wire [3:0] a,
3     input  wire [3:0] b,
4     output wire [4:0] sum
5 );
6
7     assign sum = a + b;
8
9 endmodule
```

The testbench connects its signals to the DUT ports.

```
1 adder dut (
2     .a(a),
3     .b(b),
4     .sum(sum)
5 );
```

Named port connections improve readability and reduce the risk of connecting signals to incorrect ports.

4 Generating Input Stimulus

Stimulus is the set of input values applied to the DUT during simulation.

The `initial` block is commonly used to apply a sequence of test vectors.

4.1 Example

```
1 initial begin
2     a = 4'd2;
3     b = 4'd3;
4
5     #10;
6     a = 4'd5;
7     b = 4'd4;
8
```

```
    #10;
    a = 4'd7;
    b = 4'd8;

    #10 $finish;
end
```

The delay operator #10 advances simulation time by 10 time units before the next statement executes.

For reliable testbenches, allow sufficient time for the DUT response before checking outputs.

5 Complete Example: Adder Testbench

5.1 DUT Code

```
module adder (
    input  wire [3:0] a,
    input  wire [3:0] b,
    output wire [4:0] sum
);

    assign sum = a + b;

endmodule
```

5.2 Testbench Code

```
`timescale 1ns/1ps

module tb_adder;

    reg  [3:0] a, b;
    wire [4:0] sum;

    // Instantiate DUT
    adder dut (
        .a(a),
        .b(b),
        .sum(sum)
    );

    initial begin
        $monitor("Time=%0t a=%0d b=%0d sum=%0d",
                 $time, a, b, sum);

        a = 4'd2;
        b = 4'd3;
        #10;
```

```
22
23          a = 4'd5;
24          b = 4'd4;
25          #10;
26
27          a = 4'd15;
28          b = 4'd1;
29          #10;
30
31          $finish;
32      end
33
34  endmodule
```

5.3 Expected Results

Input A	Input B	Sum
2	3	5
5	4	9
15	1	16

The sum output is 5 bits wide because adding two unsigned 4-bit values may produce a 5-bit result.

6 Clock Generation

Sequential circuits require a clock signal. A testbench can generate a periodic clock using an `always` block.

6.1 Example

```
1  reg clk;
2
3  initial clk = 1'b0;
4
5  always #5 clk = ~clk;
```

With a delay of 5 time units:

- The clock toggles every 5 time units.
- The clock period is 10 time units.
- The clock frequency is 100 MHz when the time unit is 1 ns.

This clock generator is intended for simulation.

7 Reset Generation

Reset initializes a sequential design to a known state before normal operation begins.

7.1 Example: Synchronous Reset Stimulus

```
reg clk;
reg rst;

initial clk = 1'b0;
always #5 clk = ~clk;

initial begin
    rst = 1'b1;

    repeat (2) @(posedge clk);

    rst = 1'b0;
end
```

The testbench asserts reset initially and deasserts it after two rising clock edges.

For a synchronous-reset DUT, reset is sampled on the active clock edge. In practical testbenches, reset deassertion should be coordinated carefully to avoid races with the DUT's clocked processes.

8 Monitoring Simulation Results

Verilog provides system tasks for observing simulation behavior.

8.1 $display

Prints values when the statement executes.

```
initial begin
    a = 4'd3;
    b = 4'd4;
    #1;

    $display("Sum = %0d", sum);
end
```

8.2 $monitor

Prints the specified values whenever one of the listed arguments changes.

```
initial begin
    $monitor("Time=%0t a=%d b=%d sum=%d",
             $time, a, b, sum);
end
```

8.3 $finish

Terminates the simulation.

```
initial begin
    #100;
    $finish;
end
```

Use `$finish` to prevent a testbench from running indefinitely when all tests are complete.

9 Self-Checking Testbench

A self-checking testbench compares the actual DUT output with the expected result automatically.

9.1 Example

```
module tb_adder_check;

    reg  [3:0] a, b;
    wire [4:0] sum;

    adder dut (
        .a(a),
        .b(b),
        .sum(sum)
    );

    initial begin
        a = 4'd2;
        b = 4'd3;
        #1;

        if (sum !== 5'd5)
            $display("TEST FAILED");
        else
            $display("TEST PASSED");

        a = 4'd15;
        b = 4'd1;
        #1;

        if (sum !== 5'd16)
            $display("TEST FAILED");
        else
            $display("TEST PASSED");

        $finish;
    end

endmodule
```

The case inequality operator `!==` also detects unknown (`X`) and high-impedance (`Z`) bits.

A robust regression testbench should count failures and report a final pass/fail summary.

10 Blocking and Nonblocking Assignments

Testbench stimulus often uses blocking assignments to apply inputs in a defined procedural sequence.

```
initial begin
    a = 1'b0;
    b = 1'b0;

    #10 a = 1'b1;
    #10 b = 1'b1;
end
```

For clocked RTL, nonblocking assignments are generally preferred:

```
always @(posedge clk) begin
    q <= d;
end
```

Testbenches must avoid driving or sampling signals in a way that creates races with DUT processes. In SystemVerilog, clocking blocks provide a structured way to coordinate stimulus and sampling.

11 Testbench Modeling Guidelines

- Keep testbench code separate from synthesizable design RTL.
- Instantiate the DUT using clear named port connections.
- Apply reset before testing normal operation.
- Test normal, boundary, and invalid input cases.
- Use `$monitor` or `$display` to observe important signals.
- Prefer self-checking tests over visual inspection of waveforms alone.
- Avoid race conditions between testbench stimulus and clocked DUT processes.
- Use appropriate delays and time units.
- Ensure the simulation terminates after the planned tests complete.
- Use waveform dumping when detailed debugging is required.

12 Summary

- A testbench verifies the functional behavior of a DUT through simulation.
- The basic structure includes DUT instantiation, stimulus generation, observation, and checking.
- Clock and reset generators are important for sequential designs.
- Self-checking testbenches compare actual outputs with expected values.
- Good synchronization prevents race conditions and improves verification reliability.

Key takeaway: A good testbench does more than apply inputs. It checks expected behavior, covers important corner cases, and reports failures clearly.